

Aula 42 - APIs REST

O Que É Uma API?

- **Significado:** Interface de Programação de Aplicativos.
- **Definição:** É um conjunto de regras ou protocolos que permitem que aplicativos de software se comuniquem entre si para trocar dados, recursos e funcionalidades.
- **Visão da IBM:** APIs são contratos que definem como sistemas trocam dados, impulsionando desde aplicativos bancários até redes sociais. Elas abstraem a complexidade, acelerando o desenvolvimento e permitem focar na especialidade do negócio enquanto se terceiriza o resto. Segundo a empresa, *"Sem APIs, não há transformação digital"*.

Definição Técnica

- **Acrônimo:** *Application Programming Interface*.
- **Funcionamento:** Atua como um intermediário padronizado que funciona na base de um "contrato": se o sistema A envia X no formato Y, o sistema B responde com Z.
- **Exemplo:** Um aplicativo de clima acessa os dados de um servidor via API; o aplicativo não precisa saber como os dados foram coletados, apenas como solicitá-los.

Anatomia de uma Solicitação API

O processo ocorre em milissegundos seguindo as etapas abaixo:

1. **Cliente envia uma requisição:** A aplicação faz um pedido à API (Exemplo: `GET /produtos`).
2. **Validação:** A API valida, traduz e roteia o pedido ao servidor correto.
3. **Processamento:** O servidor processa e retorna os dados para a API.
4. **Resposta:** A API organiza e formata a resposta em um formato leve (como JSON) e a entrega ao cliente.

Tipos de APIs

- **Públicas (Open APIs):** Acessíveis a todos (Exemplos: Google Maps, X/Twitter).
- **Privadas (Internal APIs):** Uso exclusivo dentro de uma empresa.
- **De Parceiros:** Integração controlada e restrita entre empresas (Exemplo: sistemas de pagamento).

Benefícios-Chave

- **Inovação mais rápida:** Reutilização de funcionalidades existentes (Exemplo: botão de "login com Facebook").
- **Escalabilidade:** Serviços independentes crescem sem travar o sistema como um todo.
- **Experiência do usuário:** Dados integrados geram aplicativos mais ricos (Exemplo: Uber utilizando a API do Google Maps).
- **Monetização:** Empresas vendem o acesso às suas próprias APIs (Exemplo: APIs de pagamentos).
- **Redução de custos:** Um e-commerce pode usar a API dos Correios para fretes e outra para pagamentos, liberando tempo e reduzindo custos operacionais.

Segurança em APIs

- **Chaves de API:** Identificam quem está acessando o sistema.
- **Tokens OAuth:** Permissões temporárias (Exemplo: permitir que o app leia e-mails, mas não os envie).
- **Criptografia:** Segurança no tráfego de dados utilizando o protocolo HTTPS.
- **Gateways:** Funcionam como firewalls para controlar o tráfego e bloquear ameaças.

Casos Reais de Uso

- **Bancos:** Utilizadas para PIX, consulta de saldos e investimentos.
- **Saúde:** Integração de dados entre prontuários médicos e laboratórios.
- **IoT (Internet das Coisas):** Sensores que enviam dados para a nuvem.

REST e RESTful

O que é REST?

- **Acrônimo:** *REpresentational State Transfer*.
- **Natureza:** É um **estilo arquitetural** (e não um protocolo), criado por Roy Fielding no ano de 2000.
- **Função:** Define um conjunto de regras para a comunicação entre sistemas utilizando o protocolo HTTP.
- **RESTful:** É a denominação dada à aplicação prática de todos os padrões e restrições do REST.

Os 6 Princípios do REST

1. **Cliente-Servidor:** Separação clara de responsabilidades entre quem consome (cliente) e quem armazena os dados (servidor). Isso confere portabilidade ao sistema, permitindo usar React na Web e React Native no smartphone, por exemplo.
2. **Stateless:** O servidor não armazena sessões. Cada requisição (*request*) feita pelo cliente deve conter absolutamente todas as informações necessárias para que o servidor entenda e responda (*response*). O servidor não "lembra" se o cliente foi autenticado na requisição anterior; portanto, os dados de sessão/autenticação devem ser enviados sempre.
3. **Cacheable:** As respostas do servidor devem deixar explícito se o recurso pode ou não ser armazenado em cache pelo cliente. O armazenamento em cache (no cliente ou no servidor) visa melhorar o desempenho do usuário e aumentar a escalabilidade do sistema.
4. **Interface Uniforme:** Manter um padrão fixo e coerente na construção da interface para que não seja confusa ao cliente. Envolve: uso correto dos verbos HTTP, endpoints coerentes (ex: todos no plural), utilização de apenas uma linguagem de comunicação (JSON) e a obrigatoriedade de sempre enviar uma resposta ao cliente.
5. **Sistema em Camadas:** O cliente acessa um endpoint sem precisar conhecer a complexidade interna ou por quantas camadas a requisição passa. Não se deve assumir conexão direta; pode haver intermediários (proxies, gateways) no loop de comunicação. A API deve ser projetada de forma que nem cliente nem servidor saibam se estão falando com a aplicação final ou com um intermediário.
6. **Código Sob Demanda (Opcional):** Geralmente as APIs enviam recursos estáticos, mas há casos onde as respostas podem conter código executável (como scripts ou miniaplicativos Java) para rodar no cliente apenas quando solicitado.

Referências e Exercício

Referências Bibliográficas

- IBM - O que é uma API? (<https://www.ibm.com/br-pt/think/topics/api>)
- IBM - O que é uma API REST? (<https://www.ibm.com/br-pt/think/topics/rest-apis>)

Exercício Proposto

- Pesquisar e implementar mais duas APIs gratuitas em uma linguagem de programação de sua preferência.
- Entregar a atividade junto com a implementação da API do Gemini.